

```

#include <stdio.h>
#include <string.h>

#define MAX_FRAMES 10
#define MAX_PAGES 50

//
-----
----- // Utility: print current
frame state
//
-----

void printFrames(int
frames[], int num_frames) { printf("[");
for (int i = 0; i < num_frames; i++) {
if (frames[i] == -1)
printf(" -");
else
printf(" %d", frames[i]);
}
printf(" ]");
}

//
-----
----- // 1. FIFO Page
Replacement
//
-----

void fifo(int pages[], int n,
int num_frames) { int frames[MAX_FRAMES];
int page_faults = 0;
int next = 0; // pointer to oldest frame for (int i = 0; i <
num_frames; i++) frames[i] = -1; printf("\n--- FIFO Page

```

```
Replacement ---\n");
```

```
for (int i = 0; i < n; i++) {  
    int page = pages[i];  
    int found = 0;
```

```
    // Check if page is already in frames for (int j = 0; j <  
    num_frames; j++) { if (frames[j] == page) { found = 1; break;  
    } }
```

```
    if (!found) {  
        frames[next] = page;  
        next = (next + 1) % num_frames;  
        page_faults++;  
    }
```

```
    printf("Page %d -> ", page);  
    printFrames(frames, num_frames);  
    printf("%s\n", found ? "" : " *");  
}
```

```
printf("Total Page Faults (FIFO): %d\n", page_faults); }
```

```
// _____
```

```
// 2. Optimal Page Replacement
```

```
// _____
```

```
void optimal(int pages[], int n, int num_frames) {  
    int frames[MAX_FRAMES];  
    int page_faults = 0;
```

```
    for (int i = 0; i < num_frames; i++) frames[i] = -1;
```

```
    printf("\n--- Optimal Page Replacement ---\n");
```

```
    for (int i = 0; i < n; i++) {  
        int page = pages[i];  
        int found = 0;
```

```
        // Check if page is already in frames
```

```

for (int j = 0; j < num_frames; j++) {
if (frames[j] == page) { found = 1; break; }
}

if (!found) {
// Find empty slot first
int empty = -1;
for (int j = 0; j < num_frames; j++) {
if (frames[j] == -1) { empty = j; break; } }

if (empty != -1) {
frames[empty] = page;
} else {
// Find the page used farthest in the future int replace = -1, farthest
= -1;
for (int j = 0; j < num_frames; j++) {
int next_use = n; // assume never used again for (int k = i + 1; k < n;
k++) {

                if (pages[k] == frames[j]) { next_use = k; break; }
            }
if (next_use > farthest) {
farthest = next_use;
replace = j;
}
}
frames[replace] = page;
}
page_faults++;
}

printf("Page %d -> ", page);
printFrames(frames, num_frames);
printf("%s\n", found ? "" : " *");
}

printf("Total Page Faults (Optimal): %d\n", page_faults); }

// _____

```

// 3. LRU Page Replacement

```
// _____  
void lru(int pages[], int n, int num_frames) {  
    int frames[MAX_FRAMES];  
    int last_used[MAX_FRAMES]; // tracks the time each frame was last used  
    int page_faults = 0;  
  
    for (int i = 0; i < num_frames; i++) {  
        frames[i] = -1;  
        last_used[i] = -1;  
    }  
  
    printf("\n--- LRU Page Replacement ---\n");  
  
    for (int i = 0; i < n; i++) {  
        int page = pages[i];  
        int found = 0, found_idx = -1;  
  
        // Check if page is already in frames  
        for (int j = 0; j < num_frames; j++) {  
            if (frames[j] == page) { found = 1; found_idx = j; break; } }  
  
        if (found) {  
            last_used[found_idx] = i; // update last used time } else {  
            // Find empty slot first  
            int empty = -1;  
            for (int j = 0; j < num_frames; j++) {  
                if (frames[j] == -1) { empty = j; break; } }  
  
            if (empty != -1) {  
                frames[empty] = page;  
                last_used[empty] = i;  
            } else {  
                // Find the least recently used frame  
                int lru_idx = 0;  
                for (int j = 1; j < num_frames; j++) { if (last_used[j] < last_used[lru_idx])  
                    lru_idx = j; }  
                frames[lru_idx] = page;  
            }  
        }  
    }  
}
```

```
last_used[lru_idx] = i;
}
page_faults++;
}
```

```
printf("Page %d -> ", page);
printFrames(frames, num_frames);
printf("%s\n", found ? "" : " *");
}
```

```
printf("Total Page Faults (LRU): %d\n", page_faults); }
```

```
// _____
```

```
// Main
```

```
// _____
```

```
int main() {
    int n, num_frames;
    int pages[MAX_PAGES];
```

```
    printf("Enter number of pages: ");
    scanf("%d", &n);
```

```
    printf("Enter page reference string:\n");
    for (int i = 0; i < n; i++) scanf("%d", &pages[i]);
```

```
    printf("Enter number of frames: ");
    scanf("%d", &num_frames);
```

```
    fifo(pages, n, num_frames);
    optimal(pages, n, num_frames);
    lru(pages, n, num_frames);
    return 0;
```

```
}
```

OUTPUT:

```
Output
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Total Page Faults (FIFO): 10

--- LRU Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [0 1 2]
Page 0 -> [1 2 0]
Page 3 -> [2 0 3]
Page 0 -> [2 3 0]
Page 4 -> [3 0 4]
Page 2 -> [0 4 2]
Page 3 -> [4 2 3]
Page 0 -> [2 3 0]
Page 3 -> [2 0 3]
Total Page Faults (LRU): 9

--- Optimal Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [0 4 3]
Page 3 -> [0 4 3]
Total Page Faults (Optimal): 7
```